ATTENTION IS ALL YOU NEED

A complete, simple guide for a Class 10 student

THE BIG IDEA A Transformer lets each word look at other words, work out which information may help, and combine that information. It does this with numbers and learned rules.

Based on the research paper by Ashish Vaswani and seven co-authors

Presented at NeurIPS 2017 | arXiv:1706.03762

A learning report: everyday examples first, real mathematics next, engineering details after that.

Prepared as a study guide. Toy numbers are used for teaching and are not measured values from a real Transformer.

How to use this guide

You do not need to know advanced maths to begin. Start with the story and the simple examples. When a formula appears, read the words around it first. The formula is a short way to write down an idea that has already been explained.

- MAIN EXPLANATION: written in simple words, suitable for a Class 10 student.
- EXTRA KNOWLEDGE: technical facts useful for a college student or software engineer.
- TOY EXAMPLE: small made-up numbers chosen so we can calculate by hand.
- PAPER FACT: a fact reported in the original research paper. These are marked and separated from teaching examples.

LEARNING RULE Understand what each step does before trying to memorise its name or formula.

What you should be able to explain at the end

- What a token and embedding are.
- Why a model needs numbers and information about word order.
- What Query, Key and Value mean in attention.
- How scores become attention weights and how values are combined.
- What the Transformer encoder and decoder do.
- Why the decoder uses a mask.
- Why the Transformer was useful in 2017 and what its limits are.
- How the main steps appear as tensor shapes in code.

Contents

Part	What you will learn
Part 1	The problem the paper wanted to solve
Part 2	The small amount of computer science we need
Part 3	Attention, explained from zero
Part 4	The complete Transformer: encoder and decoder
Part 5	How the original model was trained and tested
Part 6	Engineering: shapes, computation and common bugs
Part 7	What changed after 2017
Part 8	Revision, quiz, answers and glossary
Appendix	Formula sheet and primary sources

PART 1 — The problem the paper wanted to solve

1. What problem were the researchers trying to solve?

Computers are good at following exact instructions. Human language is harder because the same word can mean different things in different sentences, word order matters, and a useful clue can be far away from the word we are trying to understand.

In 2017, one important job was machine translation: taking a sentence in one language and producing the same meaning in another language. For example, an English sentence goes in and a French or German sentence comes out.

```
English sentence → computer model → German sentence
```

The research team wanted a model that could learn how parts of a sentence relate to one another and could be trained more efficiently than models that process words in a long chain.

2. Imagine a group project at school

Imagine your class must translate a short paragraph. Each student is responsible for understanding one word, but students may ask classmates for clues. The word “it” may need information from an earlier noun. A verb may need to know who performed the action. The sentence makes more sense when useful clues are combined.

A Transformer does something with a similar purpose, but it does not have real students or human thoughts inside it. It uses vectors (lists of numbers), calculations and rules learned from examples.

IMPORTANT The classroom story is an analogy. The model does not understand a sentence exactly as a person does. It learns numerical patterns that can help it produce useful language.

3. What came before the Transformer?

RNN: a model that reads one step at a time

An RNN is a Recurrent Neural Network. It reads a sequence in order. After processing one token, it carries information forward to the next token.

```
Token 1 → Token 2 → Token 3 → Token 4
```

This is like passing a note down a line. The fourth person may need to wait for information to pass through the earlier people. The model has a chain of steps, which limits how much of one training example can be processed at the same time.

LSTM and GRU

LSTM and GRU are improved types of RNN. They have extra controls that help them keep or forget information. They can work well, but they still read through a sequence step by step.

CNN: using small windows

A Convolutional Neural Network can look at several nearby tokens together. This can be easier to parallelise, but a small window may not connect two far-apart words in one layer. Several layers may be needed to pass information over a long distance.

4. The Transformer idea

The Transformer uses attention as its main way of mixing information between positions in a sequence. A word position can directly compare itself with other positions instead of sending all information through a long recurrent chain. Many of these comparisons can be done together using matrix calculations.

THE PAPER IN ONE SENTENCE The 2017 paper introduced an encoder-decoder model built around attention, without recurrent or convolutional layers in the main sequence-processing parts.

5. What does “sequence-to-sequence” mean?

A sequence is an ordered list. A sentence is a sequence of tokens. Sequence-to-sequence means turning one ordered list into another ordered list, such as English text into German text.

Task	Input → output example
Translation	English sentence → German sentence
Summarisation	Long article → shorter summary
Question answering	Question and context → answer

The 2017 paper mainly tested machine translation. Summarisation and question answering are useful modern examples of sequence tasks, but they are not the paper’s main experiments.

Check yourself

- Why is translating language difficult for a computer?
- What is the main problem with the step-by-step chain of a basic RNN?
- What is the key change proposed by the Transformer?

PART 2 — The small amount of computer science we need

6. AI, machine learning and deep learning

Artificial intelligence (AI) is the broad area of building computers that can perform tasks we associate with intelligent behaviour. Machine learning is one way to build such systems: give a program examples and let it learn patterns. Deep learning is a type of machine learning that uses neural networks with many layers.

```
AI ⊃ Machine Learning ⊃ Deep Learning
```

The symbol means “contains” in this diagram: machine learning is one part of AI, and deep learning is one part of machine learning.

7. What is a neural network?

A neural network is a mathematical function with adjustable numbers. During training, the computer changes many of those numbers so its answers become better on the examples it is learning from.

FORMULA $y = w \times x + b$

This tiny example has an input x, a weight w, a bias b and an output y. A real Transformer has many more numbers and layers.

Try the numbers $x = 3$, $w = 2$ and $b = 1$. Then $y = 2 \times 3 + 1 = 7$. The example is simple, but it shows how inputs and adjustable numbers combine to create an output.

8. Text must be changed into tokens

Computers store and calculate with numbers. Before text can be used by a neural network, a tokenizer breaks it into small pieces called tokens. A token may be a full word, part of a word, punctuation, or a special marker.

```
Sentence:  I love cats.
Teaching tokens:  [I] [love] [cats] [.]
```

These are teaching tokens only. A real tokenizer may split the sentence differently.

Word people see	Possible token idea	Remember
cat	[cat]	A complete word may be one token.
playing	[play] [ing]	A word may be split into parts.
Hello!	[Hello] [!]	Punctuation may be a separate token.

Word people see	Possible token idea	Remember
veryrareword	several smaller pieces	The exact split depends on the tokenizer.

9. Token IDs: labels for tokens

After splitting the text, the tokenizer gives every token an integer ID from a vocabulary. A vocabulary is the list of token pieces the tokenizer knows about.

```
I     → 10
love  → 25
cats  → 81
.     → 7

I love cats. → [10, 25, 81, 7]
```

DO NOT MIX THESE UP An ID is just a label. The number 81 does not mean that “cats” has a value or meaning of 81. It tells the program which token to look up.

10. Why use smaller word pieces?

A language has an enormous number of words, names and spellings. It would be difficult to store every possible word as a separate entry. Subword methods such as Byte Pair Encoding (BPE) reuse common pieces, helping the model handle many words it has not seen as complete words before.

PAPER FACT: the original paper used about 37,000 shared BPE tokens for its English-to-German setup and a 32,000 word-piece vocabulary for English-to-French. [1]

11. Vectors: lists of numbers

A vector is an ordered list of numbers. For example, [0.2, 0.7, -0.1, 0.5] has four numbers, so it has four dimensions. The values in a real model are learned; they are not hand-written definitions of words.

```
Toy vector for “cats”: [0.4, 0.8, 0.5, 0.1]
```

12. Embeddings: turning token IDs into useful vectors

An embedding is a vector that the model learns for each token. Instead of working only with the ID 81, the neural network looks up a row of numbers connected to that token in an embedding table.

```
Token ID 81 → embedding lookup → [0.4, 0.8, 0.5, 0.1]
```

The numbers are not a definition you can read like a dictionary. During training, they adjust so that the full model can use them to make better predictions.

13. Matrix: a table of numbers

A matrix is a rectangular table of numbers. If a sentence has three tokens and each token is represented by four numbers, its representations can be stored in a 3-by-4 matrix: three rows and four columns.

```
X =
[0.2  0.6  0.1  0.5]  ← token 1
[0.7  0.3  0.9  0.2]  ← token 2
[0.4  0.8  0.5  0.1]  ← token 3
Shape: 3 × 4
```

The shape tells you how large the table is. It is important when writing code because two matrices can only be multiplied in certain compatible shapes.

14. How does the model learn?

During training, the model tries to predict the right answer. It compares its prediction with the expected answer and calculates a loss, which is a number that measures how wrong the prediction was. An optimiser uses gradients to adjust the model's numbers.

```
training example → model prediction → loss → gradients → update model → repeat
```

Simple definitions

Word	Simple meaning
Parameter	A number the model can learn, such as a weight.
Loss	A number that measures how far a prediction is from the target.
Gradient	A signal showing how a small change to a parameter affects the loss.
Backpropagation	The process that calculates gradients through the network.
Optimiser	The method that uses gradients to update model parameters.
Inference	Using a trained model to make predictions, without training it.

The paper used Adam, an optimiser that keeps moving estimates of past gradients. You do not need the full Adam maths to understand attention. The important point is that model parameters change during training, then are used to make predictions during inference.

15. Softmax: scores into shares

Softmax changes a list of scores into positive numbers that add up to 1. You can read the results as shares or weights. A higher score usually gets a larger share.

FORMULA softmax(score i) = exp(score i) / sum of exp(all scores)

“exp” means e raised to the score. For now, focus on the purpose: convert scores into weights that add to 1.

Input scores	Approximate softmax weights
[2, 1, 0]	[0.665, 0.245, 0.090]
Total	0.665 + 0.245 + 0.090 ≈ 1.000

Softmax is important in attention because it helps turn matching scores into a set of weights that can be used to combine information.

PART 3 — Attention, explained from zero

16. First, the classroom story

Imagine three students are working together on a sentence. Each student has a small piece of information. One student needs help understanding a word and decides which classmates might be most useful. The student can give a lot of weight to one classmate, a smaller weight to another, and some weight to a third.

Attention does something similar with number vectors. It calculates which other positions are useful to the current position, then combines information from those positions. The model learns how to calculate these weights from training examples.

A CAREFUL WARNING Attention weights are not the same as a human explanation of meaning. They show how values are mixed in that calculation. A high weight does not prove that a word is the only reason for a model's answer.

17. The three important letters: Q, K and V

Name	Easy way to remember	What it does
Query (Q)	What am I looking for?	Represents the current position's information request.
Key (K)	How well might I match?	Used to compare a position with a query.
Value (V)	What information can I offer?	Provides the information that may be combined into the output.

These are names for different number vectors. A Query is not a real question written in English; a Key is not a word label; a Value is not a dictionary value. They are learned numerical representations.

18. Where do Q, K and V come from?

The model starts from the token representations and multiplies them by three different learned matrices. A learned matrix contains parameters adjusted during training.

FORMULA $Q = X \times W_Q$
$K = X \times W_K$
$V = X \times W_V$

X is the input representation. W_Q, W_K and W_V are three different learned matrices. Each makes a different view of the same input.

Why have three views? Because the job of finding a useful match is different from the job of carrying the information that will be shared. The model is allowed to learn separate ways to do these jobs.

19. Step one: compare a Query with Keys

For every query, the model compares it with keys from the positions it is allowed to look at. One common comparison is a dot product. For two vectors, multiply matching positions and add the results.

FORMULA For q = [1, 2] and k = [3, 4]:
q · k = (1 × 3) + (2 × 4) = 11

The dot product produces one number. In attention, that number acts as a matching score, not as a final answer or probability.

Doing this for every query and every key creates a table of scores. If there are three token positions, the table has three rows and three columns.

```
          Keys
         K1   K2   K3
Query Q1    s11  s12  s13
Query Q2    s21  s22  s23
Query Q3    s31  s32  s33
```

20. Why do we use Q times K-transpose?

The letter T means transpose: it swaps rows and columns. Transposing the Key matrix makes its shape line up with the Query matrix so matrix multiplication produces one score for each query-key pair.

```
Q shape       = 3 × 2
K shape       = 3 × 2
K-transpose   = 2 × 3
Q × K-transpose = (3 × 2) × (2 × 3) = 3 × 3
```

A 3-by-3 score table is the result. It contains nine scores: every one of the three queries is compared with all three keys.

21. Step two: divide by the square root of d-k

The key vector has a size. That size is called d-k, written d_k. If it contains 64 numbers, d_k is 64. When many numbers are added together in a dot product, the scores can grow in size. Very large scores can make softmax overly sharp and make learning harder.

FORMULA Scaled score = (Q × K-transpose) / square root(d_k)

If d_k = 64, then square root(64) = 8. A score of 16 becomes 16 / 8 = 2 before softmax.

This scaling keeps score sizes under better control. The original paper explains this using a mathematical argument about the variance of dot products. The simple explanation is that dividing by the square root of the key size helps keep scores manageable. [1]

22. Step three: turn scores into attention weights

After scaling, softmax is applied to each row of scores. Each row belongs to one query. The weights in that row add to 1, so they can be used as proportions for combining the values.

```
Example scaled scores: [2, 1, 0]
After softmax:       [0.665, 0.245, 0.090]
Meaning:             66.5%, 24.5%, 9.0% of the mix
```

The numbers above are a made-up teaching example. The weights in a real model are calculated from its learned Query and Key vectors and can change depending on the sentence and the layer.

23. Step four: use the weights to combine Values

The weights tell the model how strongly to use each Value vector. The model multiplies each Value by its weight, then adds the results. It normally combines information from several positions instead of selecting only one.

```
Weights:  0.4, 0.2, 0.4
V1:       [1, 0]
V2:       [0, 2]
V3:       [3, 1]

Output = 0.4 × V1 + 0.2 × V2 + 0.4 × V3
       = [0.4, 0] + [0, 0.4] + [1.2, 0.4]
       = [1.6, 0.8]
```

The output is a new vector. It contains a blend of information from all three Values, with more influence from the Values that received larger weights.

24. The most important attention formula

FORMULA Attention(Q, K, V) = softmax((Q × K-transpose) / square root(d_k)) × V

Read it as: compare Queries and Keys; scale the scores; turn them into weights; use the weights to mix Values.

```
Q and K  →  matching scores
matching scores → scaling → softmax weights
softmax weights + V → weighted mix → attention output
```

MEMORY LINE Q looks for a match. K helps calculate the match. V carries the information that gets mixed.

25. Full worked example with three tokens

Now put the steps together. This is a toy example with very small vectors so we can calculate it by hand. It is not the actual size of the 2017 model.

Input sequence: “I”, “love”, “cats”. Each token is represented by four numbers.

```
X =
[1  0  1  0]
[0  1  0  1]
[1  1  0  0]
Shape: 3 × 4
```

For easy arithmetic, choose these teaching projection matrices:

```
WQ = WK =
[1  0]
[0  1]
[1  0]
[0  1]
Shape: 4 × 2

WV =
[1   0]
[0   1]
[1   1]
[1  -1]
Shape: 4 × 2
```

Step A: multiply X by WQ to produce Q. The first row [1, 0, 1, 0] combines the first and third entries for the first output and the second and fourth entries for the second output. That gives [2, 0]. The other rows give [0, 2] and [1, 1]. WK is the same in this toy example, so K is the same too.

```
Q = K =
[2  0]
[0  2]
[1  1]
Shape: 3 × 2
```

Step B: use WV to make V. Multiplication gives:

```
V =
[2  1]
[1  0]
[1  1]
Shape: 3 × 2
```

Step C: compare Q with K by multiplying Q by K-transpose:

```
Q × K-transpose =
[4  0  2]
[0  4  2]
[2  2  2]
Shape: 3 × 3
```

Step D: d_k = 2, so square root(d_k) is about 1.414. Divide each score by 1.414:

```
Scaled scores ≈
[2.828  0     1.414]
[0     2.828  1.414]
[1.414  1.414  1.414]
```

Step E: apply softmax separately to each row. The approximate weights are:

```
Attention weights ≈
[0.768  0.045  0.187]
[0.045  0.768  0.187]
[0.333  0.333  0.333]
```

Notice each row adds up to approximately 1. The first query gives the largest share to the first Value. The second query gives the largest share to the second Value. The third query gives equal shares in this toy setup because its scores are equal.

Step F: multiply the attention-weight matrix by V to combine the Values:

```
Output ≈
[1.768  0.955]
[1.045  0.232]
[1.333  0.667]
Shape: 3 × 2
```

WHAT DID WE JUST DO? We began with three token representations. We compared them, created weights, mixed their Values, and created a new two-number representation for each token position.

26. Why can the output be a different size?

Our toy input had four numbers per token, but each attention head made an output with two numbers. That is allowed. In the real Transformer, the outputs of several heads are joined together and then passed through a learned matrix to return to the main model size.

27. What is self-attention?

Self-attention means that Queries, Keys and Values are created from the same sequence. In “I love cats”, the representation at “love” can use information from “I” and “cats”, as well as from “love” itself, when the model is allowed to see all positions.

```
I     can compare with: I, love, cats
love  can compare with: I, love, cats
cats  can compare with: I, love, cats
```

The word does not literally look with eyes. This is just a simple way to describe a mathematical process.

28. What is a contextual representation?

A contextual representation is a token vector that has been updated using information from surrounding tokens. The word “bank” in “river bank” appears in a different context from “bank” in “deposit money at a bank”. A Transformer can build different representations for those uses because the surrounding information differs.

The model’s learned parameters and the full network affect the result. Attention by itself does not guarantee that the model will always interpret every sentence correctly.

29. Multi-head attention: more than one view

A single attention calculation has one set of Query, Key and Value projections. Multi-head attention runs several attention calculations in parallel, each with its own learned projections. Then it joins the head outputs and uses another learned matrix to combine them.

FORMULA head i = Attention(Q × WQ_i, K × WK_i, V × WV_i)
MultiHead = Join(head 1, ..., head h) × WO

Each head can learn a different way to compare and mix information. Researchers sometimes find heads with patterns that look related to syntax or long-distance links, but the paper does not give each head a fixed human-made job.

30. Multi-head dimensions in the original base model

Quantity	Original base model	Simple meaning
d_model	512	The main vector size for each token.
Number of heads	8	Eight attention calculations run in parallel.
d_k per head	64	Query/Key size for each head.
d_v per head	64	Value size for each head.
Combined head size	8 × 64 = 512	The joined result returns to the 512-wide model representation.

The model size is split across eight heads: 512 divided by 8 equals 64 numbers per head for Keys and Values. Each head is not a separate full 512-wide copy of the model in the base configuration.

Check yourself

- What job does Q do? What job does K do? What job does V do?
- Why are attention weights not the same as the final output?
- Why do the weights for one query add up to 1 after softmax?
- What does multi-head attention add to the model?

PART 4 — The complete Transformer: encoder and decoder

31. Attention also needs word order

Imagine these two sentences: “The dog chased the cat” and “The cat chased the dog”. They use nearly the same words, but the order changes who did the action. A model therefore needs information about where tokens occur in a sentence.

Attention compares representations, but the original Transformer adds positional encodings so the model gets information about token positions.

32. Positional encoding in simple words

A position vector is a list of numbers linked to a token’s position. The paper uses sine and cosine maths to create these position values. The same rule can create position patterns without learning a separate position vector for every possible place.

FORMULA PE(pos, 2i) = sin(pos / 10000^(2i / d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i / d_model))

This looks complicated, so first understand the parts: pos is the token position; i chooses a pair of vector dimensions; d_model is the vector size; sin and cos are repeating mathematical functions.

Some position dimensions change quickly as position increases; others change slowly. Together they give different positions different number patterns. In the original model, the position vector has the same size as the token embedding so the two can be added.

33. A tiny position example

For a made-up example with d_model = 4 and position pos = 1, the first pair of values is sin(1) and cos(1), which are about 0.8415 and 0.5403. For the next pair, the formula uses a slower change and gives values close to 0.0100 and 0.99995.

Toy position vector for position 1 ≈ [0.8415, 0.5403, 0.0100, 0.99995]

These are demonstration values for a four-number vector. The real base Transformer uses 512 dimensions.

34. Add token and position information

Token embedding: [0.2, 0.5, 0.3, 0.7]
Position vector: [0.8, 0.5, 0.01, 1.0]
Add them: [1.0, 1.0, 0.31, 1.7]

Each token gets its own embedding plus its own position vector. This tells the model both which token it is processing and where that token appears.

35. The encoder: reading the source

The encoder processes the input sentence and makes a final representation for every source token. For translation, the source might be the English sentence. The model then uses these representations to help make the target sentence.

The original base Transformer contains six encoder layers stacked one after another. Each layer has two main parts: self-attention and a feed-forward network.

```
Encoder layer:
input representations
      ↓
self-attention: mix information between positions
      ↓
Add + LayerNorm
      ↓
feed-forward network: transform each position
      ↓
Add + LayerNorm
      ↓
output to the next layer
```

36. Residual connection: keep a direct path

A residual connection adds the input of a layer part to its output. In simple notation, it is “input + transformed input”. This provides a direct path for information and helps deep networks train.

FORMULA Layer result = LayerNorm(x + Sublayer(x))

In the original Transformer, the input x is added to the sublayer output, then LayerNorm is applied. This order is called post-norm.

37. Layer normalization

Layer normalization, often called LayerNorm, adjusts a vector using its mean and spread, then applies learned scale and shift values. The aim is to keep the numbers in a useful range for learning. You can first think of it as a way to standardise a vector before the next stage.

FORMULA LayerNorm(x) = gamma × (x - mean) / sqrt(variance + epsilon) + beta

Here gamma and beta are learned values, and epsilon is a tiny number used to avoid division problems. You do not need to calculate LayerNorm by hand to understand the main attention idea.

38. The feed-forward network (FFN)

After attention has mixed information between token positions, a feed-forward network transforms the vector at each position. It uses the same learned network for each position in the layer, but each position’s input values may be different.

FORMULA FFN(x) = ReLU(x × W1 + b1) × W2 + b2

The first matrix expands the vector, ReLU changes negative values to zero, and the second matrix brings it back to the main vector size.

39. What is ReLU?

```
ReLU(number) = the bigger of 0 and that number
[-2, 3, -1, 5] → [0, 3, 0, 5]
```

PAPER FACT: in the base model, d_model = 512 and the middle FFN size d_ff = 2048. So the FFN expands from 512 numbers to 2048 numbers and then returns to 512 numbers. [1]

REMEMBER THE DIFFERENCE Attention lets positions exchange information. The FFN transforms each position separately. Both are needed in the original Transformer layer.

40. The decoder: building the answer

The decoder generates the target sequence. In translation, it helps produce the German sentence from the English source and the target words generated so far. The original base Transformer has six decoder layers.

Each decoder layer has three main parts: masked self-attention, encoder-decoder attention (also called cross-attention), and a feed-forward network. Residual connections and LayerNorm are used around these parts.

41. Why does the decoder need a mask?

When generating a sentence from left to right, the model must not use future answer tokens to predict the current token. If it were predicting “love”, it must not get to peek at the future word “cats”. That would be cheating during training.

```
Target positions:      I   love   cats
When predicting I:     see  -     -
When predicting love:  see  see   -
When predicting cats:  see  see   see
```

The mask blocks future positions. The original method gives blocked positions a score of negative infinity before softmax. The exponential of negative infinity becomes zero, so the blocked positions receive zero attention weight.

MASK MEMORY LINE A causal mask means: use the past and present, but do not look at future target tokens.

42. Training uses shifted answers

During training, the correct target sentence is available. The decoder receives a version shifted one token to the right, often starting with a special start marker. It learns to predict the next token at each position. A causal mask prevents a position from seeing the answer it is supposed to predict or anything beyond it.

```
Decoder input:  <START>  I     love
Target to learn: I        love  cats
```

The illustration is simplified. Real tokenizers and training pipelines define special tokens and padding details explicitly.

43. Cross-attention: the decoder looks back at the source

After masked self-attention, the decoder also needs the source sentence. In cross-attention, the Query comes from the decoder, while the Keys and Values come from the encoder output.

```
Q ← decoder representations
K ← encoder output
V ← encoder output
```

This lets the decoder find source information useful for the target token it is producing. For example, while producing a translated word, the decoder can use relevant information from the original sentence.

44. From decoder output to a token

After the decoder stack, a linear layer makes one raw score for every token in the target vocabulary. These raw scores are called logits. Softmax turns them into probabilities. The model then selects or searches among possible next tokens.

```
decoder output → linear layer → logits → softmax → probabilities → next token
```

A logit is not a probability. Logits may be positive or negative and do not need to add up to 1. Probabilities after softmax are between 0 and 1 and add up to 1 for the list being normalised.

45. The complete architecture at a glance

```
SOURCE SENTENCE
    ↓
Tokenizer → Token IDs → Embeddings + Position
    ↓
ENCODER (6 layers in the base model)
  Self-attention → Add/Norm → FFN → Add/Norm
    ↓
Encoded source representations
    │
TARGET START / PREVIOUS TARGET TOKENS
    ↓
Embeddings + Position
    ↓
```

```
DECODER (6 layers in the base model)
  Masked self-attention → Add/Norm
  Cross-attention to encoder output → Add/Norm
  FFN → Add/Norm
    ↓
Linear layer → Softmax → next-token probabilities
```

46. Training versus generating

During training	During autoregressive generation
The correct target sentence is already known.	The future target words are not known yet.
Many target positions can be processed together because the mask blocks future information.	The model generates one next token, then uses it to produce the next one.
The model compares predicted tokens with the known targets and updates its parameters.	The trained model normally uses its parameters without updating them.

VERY IMPORTANT Transformer training is highly parallelisable, but ordinary autoregressive generation still proceeds one token at a time.

Check yourself

- Why does the model need position information?
- What do residual connections add?
- What is the job of the FFN?
- Why does the decoder need a causal mask?
- How is cross-attention different from self-attention?

PART 5 — How the original model was trained and tested

47. What counts as evidence that a model works?

The researchers did not only describe an idea. They trained the Transformer and compared its results with earlier machine-translation systems. They also changed parts of the model to see how those choices affected results.

48. The translation data

Task	Training data reported in the paper	Token vocabulary
English → German	About 4.5 million sentence pairs	About 37,000 shared BPE tokens
English → French	About 36 million sentences in the dataset, as described in the paper	32,000 word-piece vocabulary

A sentence pair means the same content written in two languages. The model learns from many examples of a source sentence and its target translation. [1]

49. Why group examples by length?

The paper grouped sentences of similar lengths into batches. A batch is a group of training examples processed together. The authors describe batches of approximately 25,000 source tokens and 25,000 target tokens. Token-based batching helps control the amount of work and memory used by a batch.

50. Hardware and training time

Model setup	Reported setting
Hardware	One machine with 8 NVIDIA P100 GPUs
Base model	About 0.4 seconds per training step; 100,000 steps; around 12 hours
Big model	About 1.0 second per training step; 300,000 steps; around 3.5 days

These numbers describe a specific 2017 experiment. They should not be treated as a promise about how long the same training would take on a different computer today. [1]

51. The original model's main settings

Setting	Base model	Big model
Encoder layers	6	6
Decoder layers	6	6

Setting	Base model	Big model
Main vector size d_model	512	1024
FFN middle size d_ff	2048	4096
Attention heads	8	16
Key/value size per head	64	64 in the usual big-model setting
Parameters	About 65 million	About 213 million
Training steps	100,000	300,000
Dropout setting	0.1	0.3 for the big-model configuration in the paper's main setup

A parameter is a number learned during training. More parameters give the model more flexibility, but also usually require more memory and computation. Bigger is not automatically better for every task.

52. What are Adam and warmup doing?

Adam is the optimiser used in the paper. The learning rate controls how large parameter updates are. The authors first increase the learning rate during a warmup period, then reduce it over time. This is a schedule, not a guarantee that every update is correct.

FORMULA Learning rate = d_model^(-0.5) × min(step^(-0.5), step × warmup^(-1.5))
Warmup steps = 4000

This is the schedule from the paper, written in compact form. The formula is useful for engineers; a beginner only needs to remember "increase at first, then gradually decrease".

Adam setting	Paper value
β1	0.9
β2	0.98
epsilon	0.000000001 (1e-9)
Warmup steps	4,000

53. Dropout: do not depend too much on the training examples

Dropout is a training technique that temporarily turns off some activations at random. It encourages the network not to depend too heavily on only one path or feature. The paper uses dropout in the model and at the embedding-plus-position stage. The base model uses a dropout rate of 0.1.

54. Label smoothing

During training, a model is encouraged to give high probability to the correct token. Label smoothing makes the target slightly less absolute: a little probability is shared with other choices rather than assigning all target probability to one answer.

FORMULA Smoothed target = (1 - epsilon) × original target + epsilon × smoothing distribution

The paper uses a label-smoothing value of 0.1. The authors report that label smoothing raised perplexity but improved translation measures such as BLEU. [1]

55. How did they measure translation quality?

BLEU

BLEU is a metric used to compare machine-generated translations with human reference translations. It checks how much short word sequences overlap, with a penalty related to translation length. It is helpful for comparing systems under the same evaluation setup, but it is not a complete measure of meaning or human understanding.

Perplexity

Perplexity measures how much probability a language model gives to the correct sequence of tokens. Lower perplexity often means the model gives higher probability to the expected tokens. In the original paper, the reported development-set perplexities use word pieces, so they should not be compared casually with numbers reported per full word.

F1

F1 combines precision and recall. Precision asks “Of the things the model predicted, how many were right?” Recall asks “Of the things that should have been found, how many did the model find?” It is used in the paper’s parsing experiment in the longer arXiv version.

56. Main translation results

Model	WMT14 English → German BLEU	WMT14 English → French BLEU
Transformer base	27.3	38.1
Transformer big	28.4	41.0 in the published NeurIPS 2017 results table

The results show that the Transformer was competitive with or better than strong systems of its time, while making more of the training computation parallelisable. They do not show that it was perfect or that all modern language systems use every original setting. [1]

A NOTE ON PAPER VERSIONS The current arXiv record (v7) reports 41.8 BLEU for English-to-French in its abstract, while the published NeurIPS 2017 results table reports 41.0. When giving a

historical number, name the version. This guide uses the published proceedings table for the main comparison. [1, 2]

57. What is beam search?

A model can give probabilities for many possible next tokens. Greedy decoding chooses the single highest-probability next token each time. Beam search keeps several promising partial sentences, expands them and then compares the candidates. The paper used a beam size of 4 and a length-penalty setting of alpha = 0.6 for translation decoding.

Beam search does not guarantee the best possible sentence. It is a practical search method that checks more than one possible continuation without testing every possible sentence.

58. What is an ablation study?

An ablation study changes one part of a system and measures what happens. It helps researchers understand which choices seem to matter. For example, they can compare one attention head with eight or change the number of layers.

Experiment reported for English-to-German development data	PPL	BLEU	What it suggests
Base: 8 heads, 6 layers, d_model 512	4.92	25.8	Reference configuration
1 attention head	5.29	24.9	One head performed worse here.
4 heads	5.00	25.5	Four heads were close but below the base BLEU.
16 heads	4.91	25.8	More heads did not automatically improve BLEU.
2 layers	6.11	23.7	A shallower model performed worse here.
4 layers	5.19	25.3	More depth helped compared with 2 layers.
8 layers	4.88	25.5	The bigger model performed well, but used more parameters.
Learned positional embeddings	4.92	25.7	Very similar to sinusoidal positions in this test.

These are development-set results from the paper's variation table, not the main test-set results. PPL is per word piece. Small differences should be interpreted carefully and in the context of the full experimental setup. [1]

59. What did the extra parsing experiment test?

Parsing means working out the grammatical structure of a sentence, such as which words form a noun phrase or verb phrase. The longer arXiv version reports additional English constituency-parsing experiments. The reported F1 results give evidence that the Transformer could be used for more than machine translation. Keep this separate from the main translation table because it is a different task and evaluation setup. [2]

60. What did attention visualisations show?

The longer arXiv version includes pictures of attention weights. Some heads show patterns that appear connected with long-distance word relationships or sentence structure. These pictures are useful evidence about how the network is operating, but they do not prove that each head follows one fixed grammar rule or that attention weights fully explain the model's reasoning. [2]

61. What can we honestly conclude?

- The Transformer showed that strong sequence-to-sequence models could be built without recurrent or convolutional sequence-processing layers.
- Training computation was easier to parallelise than in basic recurrent models.
- Attention gave positions a direct way to combine information from other positions.
- The paper tested real translation tasks and reported strong results for its time.
- The design still has costs, including the quadratic growth of full attention with sequence length.

PART 6 — Engineering: shapes, computation and common bugs

62. Why software engineers care about tensor shapes

A tensor is a number container. A single number is a tensor; a vector is a tensor; a matrix is a tensor; and a higher-dimensional block of numbers is also a tensor. In a Transformer, we need to know how many examples are in a batch, how many tokens are in each example, and how many numbers represent each token.

Symbol	Meaning
B	Batch size: number of examples processed together.
N	Sequence length: number of tokens in one example.
D or d_model	Main number of features for each token.
H	Number of attention heads.
Dh	Number of features in one head; usually D divided by H.

63. Tensor shapes in the original base model

```
Input X: [batch, sequence length, model size]
      = [B, N, 512]

After Q/K/V projection, then split heads:
Q, K, V = [B, 8, N, 64]

Attention scores:
Q × K-transpose = [B, 8, N, N]

Weights × V:
[B, 8, N, N] × [B, 8, N, 64]
= [B, 8, N, 64]

Join the 8 heads:
[B, N, 8 × 64] = [B, N, 512]
```

The shape [B, H, N, N] holds one score for each pair of token positions for each head in each example. It becomes important for memory as N grows.

64. Why full attention can use a lot of memory

If a sequence has N tokens, full attention makes an N-by-N score table. That means N squared entries per head and per example. Doubling N makes the score table about four times as large.

Sequence length N	Entries in one N × N score table
10	100
100	10,000
1,000	1,000,000
4,000	16,000,000

This quadratic growth is an important limitation of the original full self-attention mechanism. Modern systems use improved implementations and alternative attention patterns to reduce costs, but those are later developments rather than the original 2017 design.

65. Why is training parallel but generation sequential?

During training, all correct target tokens are known. A causal mask blocks forbidden future information, so the computer can calculate many target positions at once. During generation, the next token must be chosen before it can be used to help generate the token after it.

```
Training:  calculate many positions together (with the correct mask)
Generation: token 1 → token 2 → token 3 → ...
```

66. A small engineer's checklist

- Is the model size D divisible by the number of heads H?
- Does each head use Dh = D / H?
- Do the last two dimensions match for matrix multiplication?
- Are attention scores divided by the square root of Dh?
- Is softmax applied over the key positions for each query?
- Does the decoder mask block all future target positions?
- Are padding tokens handled using the correct padding mask where needed?
- Do residual branches have matching shapes?
- Is the target shifted correctly during training?
- Does generation stop when the end condition is reached?

67. Common bugs and how to reason about them

Problem	What to inspect
Matrix multiplication fails	Write out the dimensions. For (a × b)(b × c), the middle sizes must match.
Attention weights look strange	Check scores, scale, softmax axis and masks. Each allowed query row should sum to about 1.
Future target information leaks	Check the causal mask direction and whether it is applied before softmax.
Output shape changed unexpectedly	Check how the head dimension is split, transposed, joined and projected.

Problem	What to inspect
Training loss does not improve	Check data/token IDs, target shifting, loss, gradients, learning rate, masks and padding.
Results differ from the 2017 paper	Check model settings, data, tokenizer, evaluation version, training steps and decoding setup.

68. Pseudocode for attention

This is not a full program; it is a compact outline showing the order of operations. In code, exact tensor axes and mask broadcasting must be handled carefully.

```
Q = X @ WQ
K = X @ WK
V = X @ WV

scores = Q @ transpose(K)
scores = scores / sqrt(key_dimension)
if a mask is needed:
    scores = apply_mask(scores)
weights = softmax(scores, over_key_positions)
output = weights @ V
```

69. Original architecture versus a modern implementation

The original paper is a particular 2017 model. Modern libraries may use different normalisation order, position methods, activation functions, attention kernels, cache systems, numerical precision and training methods. Those differences can be good engineering choices, but a faithful reproduction of the paper must match its original settings.

PART 7 — What changed after 2017

70. The 2017 paper is a foundation, not the whole modern recipe

The original Transformer has an encoder and a decoder. After the paper, researchers developed many models that use Transformer layers in different ways. The following are later developments. They should not be described as features introduced by the original paper.

71. BERT: an encoder-focused model

BERT stands for Bidirectional Encoder Representations from Transformers. It uses an encoder-style Transformer to learn representations using context from both sides of a token. It is often used for tasks such as classification, finding answers in a passage and understanding text. BERT was published after the original Transformer, in 2018. [3]

72. GPT: a model that predicts the next token

GPT-style models use a decoder-style, masked Transformer for autoregressive text generation. The model predicts a next token, then uses the growing sequence to predict another. Many later large language models follow this broad approach, although their training data, sizes, systems and training methods can differ greatly from early GPT. [4]

73. T5: many tasks as text-in and text-out

T5 means Text-To-Text Transfer Transformer. It uses an encoder-decoder design and frames many language tasks as converting input text to output text. For example, an input paragraph can produce a summary, or an English sentence can produce a translation. It is a later work, published in 2020. [5]

Model family	Basic structure	Simple mental picture
Original Transformer (2017)	Encoder + decoder	Read a source sequence, then create a target sequence.
BERT	Encoder-focused	Build rich representations using left and right context.
GPT-style	Decoder-style causal model	Predict the next token from earlier tokens.
T5	Encoder + decoder	Turn many text tasks into text-in, text-out jobs.

74. What modern LLMs may add

Later systems may use larger training collections, different positional methods, more efficient attention, different normalisation and activation choices, instruction tuning, human-preference training, retrieval, tool calling, caching, distributed training and other engineering changes. Each should be studied as its own topic rather than assumed to be part of the 2017 paper.

75. What the 2017 paper did not prove

- It did not prove that attention solves every language problem.
- It did not show that every attention head is one fixed grammar rule.
- It did not make autoregressive text generation fully parallel.
- It did not remove the cost of full attention over very long sequences.
- It did not include the complete training and deployment recipe used by modern large language models.

76. Why the paper mattered

The paper changed the main way many language systems were built. Instead of depending on a recurrent chain to move information through the input, the Transformer used attention layers to let positions exchange information directly. That made training more parallelisable and produced strong results on translation. Many later language models built on this idea.

PART 8 — Revision, quiz, answers and glossary

77. The whole Transformer as a simple recipe

1. Break the text into tokens.
2. Turn each token into an integer ID.
3. Look up an embedding vector for each ID.
4. Add a position vector so the model receives word-order information.
5. Use encoder self-attention to mix source information.
6. Use residual connections and LayerNorm around the layer parts.
7. Use the feed-forward network to transform each position.
8. Repeat encoder layers to build stronger representations.
9. Give the decoder the start marker and previous target tokens.
10. Use masked decoder self-attention so future targets cannot be seen.
11. Use cross-attention to read the encoder output.
12. Use the decoder stack to make a representation for the next output.
13. Convert its representation into vocabulary logits and probabilities.
14. Choose a next token and repeat until the answer is finished.

78. Twenty-five revision questions

1. What is a token?
2. Why is a word not always one token?
3. What is a token ID?
4. What is an embedding?
5. What does a vector contain?
6. What is a matrix?
7. What is the difference between training and inference?
8. What does attention help the model do?
9. What does Query mean in the simple analogy?
10. What does Key do?
11. What does Value provide?
12. Why compare Q with K?
13. Why divide by the square root of d_k?
14. What does softmax do?
15. Why multiply the weights by V?

16. What is self-attention?
17. What is multi-head attention?
18. Why add positional information?
19. What does the encoder produce?
20. What does the decoder produce?
21. Why mask future target positions?
22. What is cross-attention?
23. Why can Transformer training process many positions together?
24. Why does autoregressive generation still happen one token at a time?
25. Name one important limitation of full self-attention.

79. Answer key

1. A token is a piece of text processed by the model.
2. A tokenizer can split rare or long words into smaller parts.
3. A token ID is an integer label that identifies a vocabulary token.
4. An embedding is a learned list of numbers for a token.
5. A vector contains an ordered list of numbers.
6. A matrix is a rectangular table of numbers.
7. Training adjusts model parameters; inference uses the trained parameters to make predictions.
8. Attention lets a position use information from other positions.
9. Query is a vector that helps represent what information a position is looking for.
10. Key helps calculate how well a position matches a Query.
11. Value carries the information that attention weights combine.
12. Comparing Q and K creates the scores used to calculate attention weights.
13. Scaling helps keep dot-product scores manageable when the key size is large.
14. Softmax turns scores into positive weights that add to 1 across the chosen row.
15. The weighted Values are combined to form the attention output.
16. Self-attention uses Q, K and V from the same sequence.
17. Multi-head attention runs multiple learned attention calculations and combines their outputs.
18. Word order changes meaning, so position information helps the model distinguish positions.
19. The encoder produces representations of source tokens.
20. The decoder produces representations used to predict target tokens.
21. The mask prevents the model from peeking at future answer tokens.
22. Cross-attention uses Query from the decoder and Key/Value from the encoder output.
23. The causal mask prevents future target information from leaking, so positions can be computed together during training.
24. The next generated token is needed before the model can condition on it to generate the next one.

25. Full attention creates an N-by-N score table, so its cost grows roughly with the square of sequence length.

80. Glossary of important words

Term	Simple definition
AI	The broad field of building computer systems that perform tasks linked to intelligent behaviour.
Attention	A calculation that gives different weights to information from different positions.
Autoregressive	Generating the next token using tokens already generated.
Batch	A group of examples processed together.
BERT	A later encoder-focused Transformer model.
BPE	A method that builds text tokens from reusable pieces.
Causal mask	A rule that blocks future target positions.
CNN	A neural network type that uses local windows called convolutions.
Context	Other information around the current token or task.
Cross-attention	Attention where Query comes from one source and Key/Value from another.
Decoder	The Transformer part that helps produce target tokens.
Dimension	The number of entries in a vector.
Embedding	A learned vector linked to a token ID.
Encoder	The Transformer part that creates representations of the source sequence.
FFN	A feed-forward network that transforms each token position separately.
Gradient	A value describing how a change in a parameter affects loss.
LayerNorm	A calculation that normalises values within a representation.
Logit	A raw score before softmax.
Loss	A number that measures prediction error.
Matrix	A rectangular table of numbers.
Multi-head attention	Several attention calculations with different learned projections.
Parameter	A number the model learns during training.
Positional encoding	Numbers added to embeddings to provide information about token position.
Query	One of the vectors used to calculate attention matches.
Key	One of the vectors compared with Queries.

Term	Simple definition
Value	One of the vectors mixed using attention weights.
RNN	A recurrent network whose state is updated step by step.
Softmax	A function that turns scores into positive weights that sum to 1.
Token	A piece of text processed by the model.
Tokenizer	Software that splits text into tokens and maps them to IDs.
Training	The process of adjusting model parameters using examples.
Transformer	A neural-network architecture that uses attention as a core sequence-processing mechanism.
Vector	An ordered list of numbers.

81. Formula sheet

Idea	Formula	Read it as
Query, Key, Value	Q = XWQ; K = XWK; V = XWV	Use learned matrices to make three views of the input.
Scaled dot-product attention	softmax(QK^T / √d_k)V	Compare, scale, turn into weights, combine values.
Multi-head attention	Concat(head_1, ..., head_h)W_O	Run several attention heads and combine them.
Feed-forward network	ReLU(xW_1 + b_1)W_2 + b_2	Expand, apply ReLU, shrink back.
ReLU	max(0, x)	Negative numbers become zero; positive numbers stay.
Residual + LayerNorm	LayerNorm(x + Sublayer(x))	Add original input to sublayer output, then normalise.
Positional encoding	sin/cos functions of position	Give each position a number pattern.

82. The original Transformer base model: key numbers

Item	Value
Encoder layers	6
Decoder layers	6
Main model width	512
FFN middle width	2048
Attention heads	8
Key width per head	64

Item	Value
Value width per head	64
Base dropout	0.1
Label smoothing	0.1
Warmup	4,000 steps
Adam beta1 / beta2	0.9 / 0.98
Base parameters	About 65 million
Big-model parameters	About 213 million

83. One-page final recap

PROBLEM
Basic recurrent models process tokens in a chain, making parallel training harder.

IDEA
Use attention to let positions combine useful information from other positions.

INPUT
Text → tokens → token IDs → embeddings + positional information.

ATTENTION
Q and K produce scores; softmax produces weights; weights mix V.

ENCODER
Self-attention + Add/Norm + FFN + Add/Norm, repeated 6 times in the base model.

DECODER
Masked self-attention + cross-attention + FFN, with residuals and LayerNorm.

OUTPUT
Linear layer → logits → softmax probabilities → next token.

KEY BENEFIT
More parallelisable training and direct token-to-token information paths.

KEY LIMIT
Full attention stores an N-by-N score table; autoregressive generation is still step by step.

THE DEEPEST IDEA A Transformer repeatedly updates each token's numerical representation by gathering and mixing information from other allowed token positions. The 2017 paper made attention the central mechanism in an encoder-decoder model.

APPENDIX — Sources and accuracy notes

Primary source: the 2017 paper

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser and Illia Polosukhin. “Attention Is All You Need.” NeurIPS 2017.

Published proceedings paper: https://papers.neurips.cc/paper/7181-attention-is-all-you-need

Published PDF: https://papers.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[2] Later arXiv version (v7, revised 2023): Attention Is All You Need. https://arxiv.org/abs/1706.03762

Later-model references

[3] Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” 2018. https://arxiv.org/abs/1810.04805

[4] Alec Radford and colleagues. “Improving Language Understanding by Generative Pre-Training.” 2018. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

[5] Colin Raffel and colleagues. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.” 2020. https://www.jmlr.org/papers/v21/20-074.html

About versions and teaching examples

- The published NeurIPS proceedings and later arXiv versions are not identical in every detail. The paper’s version should be named when reporting results. This guide uses the published NeurIPS 2017 table for the main English-to-French result and separately notes the arXiv abstract’s 41.8 figure.
- Toy token splits, toy embedding values, the classroom story and the small attention calculation are made for teaching. They are not outputs measured from the paper’s trained model.
- Q = “what am I looking for?”, K = “what may match?”, and V = “what information is offered?” are memory aids. Technically, Q, K and V are learned numerical projections.
- Attention weights help combine values, but should not be presented as a full explanation of every internal step in the model’s reasoning.
- The original Transformer is encoder-decoder. BERT, GPT-style models, T5 and modern large-language-model training methods came later.
- The original Transformer uses post-norm residual blocks written as LayerNorm(x + Sublayer(x)). Modern implementations may choose another order.

FINAL MEMORY SENTENCE A Transformer uses learned attention weights to mix information between token positions, then uses more layers to build representations that help it predict or transform a sequence.